

Computer Architecture

Project: MIPS 32 Processor Implementation
Single-Cycle Architecture

Problem: Sum of powers of 2 within an interval $[a, b]$

Student:

Măgerușan Alexandru

Date:

May 2, 2026

Contents

1	Introduction	2
2	Solution Algorithm	2
2.1	C++ Source Code	2
2.2	x86 Assembly Code	3
2.3	MIPS Assembly Code	4
3	Instruction Set and Control Signals	5
3.1	Instruction Classification and Datapath	5
3.1.1	R-Type Instructions (Register)	5
3.1.2	I-Type Instructions (Immediate)	6
3.1.3	J-Type Instructions (Jump)	7
4	Hardware Components and Architecture	8
4.1	Processor Block Diagram	8
4.2	IFetch.vhd	9
4.3	IDecode.vhd	11
4.4	EX.vhd	12
4.5	MEM.vhd	14
4.6	UC.vhd	15
4.7	test_env.vhd	16
5	Machine Code and Execution	19
6	Execution Trace	20
7	FPGA Synthesis	20

1 Introduction

This project presents the hardware design of a 32-bit Single-Cycle MIPS microprocessor implemented in VHDL. The system is validated by solving a specific problem: calculating the sum of powers of 2 from a given dataset, conditionally restricted to a specified interval.

2 Solution Algorithm

2.1 C++ Source Code

High-level reference implementation of the algorithm.

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int a = 2;
6      int b = 90;
7      int x[] = {0, 1, 2, 3, 256, 14, 16};
8      int n = 7;
9
10     int sum = 0;
11     int i = -1;
12
13     while (i < n - 1)
14     {
15         i++;
16         int val = x[i];
17
18         if (val < a) continue;
19         if (val > b) continue;
20         if (val <= 0) continue;
21
22         int temp = val - 1;
23         int test = val & temp;
24
25         if (test != 0) continue;
26
27         sum = sum + val;
28     }
29
30     printf("Sum:%d\n", sum);
31     return 0;
32 }
```

C++ Reference Program

2.2 x86 Assembly Code

Generated x86 code for testing the processing logic.

```
1 main:
2     push    rbp
3     mov     rbp, rsp
4     mov     DWORD PTR [rbp-12], 2
5     mov     DWORD PTR [rbp-16], 90
6     mov     DWORD PTR [rbp-64], 0
7     mov     DWORD PTR [rbp-60], 1
8     mov     DWORD PTR [rbp-56], 2
9     mov     DWORD PTR [rbp-52], 3
10    mov     DWORD PTR [rbp-48], 256
11    mov     DWORD PTR [rbp-44], 14
12    mov     DWORD PTR [rbp-40], 16
13    mov     DWORD PTR [rbp-20], 7
14    mov     DWORD PTR [rbp-4], 0
15    mov     DWORD PTR [rbp-8], -1
16    jmp     .L2
17 .L7:
18     add     DWORD PTR [rbp-8], 1
19     mov     eax, DWORD PTR [rbp-8]
20     cdqe
21     mov     eax, DWORD PTR [rbp-64+rax*4]
22     mov     DWORD PTR [rbp-24], eax
23     mov     eax, DWORD PTR [rbp-24]
24     cmp     eax, DWORD PTR [rbp-12]
25     jge     .L3
26     jmp     .L2
27 .L3:
28     mov     eax, DWORD PTR [rbp-24]
29     cmp     eax, DWORD PTR [rbp-16]
30     jle     .L4
31     jmp     .L2
32 .L4:
33     cmp     DWORD PTR [rbp-24], 0
34     jg      .L5
35     jmp     .L2
36 .L5:
37     mov     eax, DWORD PTR [rbp-24]
38     sub     eax, 1
39     mov     DWORD PTR [rbp-28], eax
40     mov     eax, DWORD PTR [rbp-24]
41     and     eax, DWORD PTR [rbp-28]
42     mov     DWORD PTR [rbp-32], eax
43     cmp     DWORD PTR [rbp-32], 0
44     je      .L6
45     jmp     .L2
46 .L6:
47     mov     eax, DWORD PTR [rbp-24]
48     add     DWORD PTR [rbp-4], eax
49 .L2:
50     mov     eax, DWORD PTR [rbp-20]
51     sub     eax, 1
52     cmp     DWORD PTR [rbp-8], eax
53     jl      .L7
54     mov     eax, 0
55     pop     rbp
56     ret
```

x86 Assembly Code

2.3 MIPS Assembly Code

Final implementation intended for execution on the MIPS processor.

```
1  .data
2      # Declare the array in memory (each number occupies 1 word = 4 bytes)
3      x: .word 0, 1, 2, 3, 256, 14, 16
4
5  .text
6  .globl main
7  main:
8      addi $1, $0, 2      # a = 2
9      addi $2, $0, 90     # b = 90
10     addi $3, $0, 7      # n = 7
11     la    $4, x          # base_addr = starting address of array x
12     add   $5, $0, $0     # sum = 0
13     add   $6, $0, $0     # i = 0
14
15 loop_start:
16     slt   $7, $6, $3     # $7 = 1 if i < n
17     beq   $7, $0, end    # If i >= n, done, jump to 'end'
18
19     # --- ADDRESS CORRECTION ---
20     sll   $9, $6, 2      # $9 = i * 4
21     add   $9, $4, $9     # $9 = address of current element
22     lw    $8, 0($9)     # val = read x[i] into $8
23
24     addi  $6, $6, 1      # i = i + 1
25
26     # Check: val < a
27     slt   $7, $8, $1     # $7 = 1 if val < a
28     bne   $7, $0, loop_start
29
30     # Check: val > b
31     slt   $7, $2, $8     # $7 = 1 if b < val
32     bne   $7, $0, loop_start
33
34     # Check: val > 0
35     slt   $7, $0, $8     # $7 = 1 if 0 < val
36     beq   $7, $0, loop_start
37
38     # Check: is it a power of 2? (val & (val - 1) == 0)
39     addi  $9, $8, -1     # $9 = val - 1
40     and   $9, $8, $9     # $9 = val & (val - 1)
41     bne   $9, $0, loop_start
42
43     # Add to sum
44     add   $5, $5, $8     # sum = sum + val
45     j     loop_start
46
47 end:
48     addi  $v0, $0, 10    # exit
49     syscall
```

MIPS Assembly Code for the processor

3 Instruction Set and Control Signals

The processor decodes a subset of 10 MIPS 32 instructions. The following table presents the binary encodings and signals generated by the Control Unit (UC).

Instruction	Opcode [31-26]	RegDst	ExtOp	ALUSrc	Branch	Br_ne	Jump	MemWrite	MemtoReg	RegWrite	ALUOp[1:0]	function [5-0]	ALUCtrl[2:0]
add	000000	1	0	0	0	0	0	0	0	1	10	100000	000 (+)
and	000000	1	0	0	0	0	0	0	0	1	10	100100	010 (&)
slt	000000	1	0	0	0	0	0	0	0	1	10	101010	011 (i)
sll	000000	1	0	0	0	0	0	0	0	1	10	000000	100 (ii)
addi	001000	0	1	1	0	0	0	0	0	1	00 (+)	X	000 (+)
lw	100011	0	1	1	0	0	0	0	1	1	00 (+)	X	000 (+)
sw	101011	0	1	1	0	0	0	1	0	0	00 (+)	X	000 (+)
beq	000100	0	1	0	1	0	0	0	0	0	01 (-)	X	001 (-)
bne	000101	0	1	0	0	1	0	0	0	0	01 (-)	X	001 (-)
j	000010	0	0	0	0	0	1	0	0	0	00	X	000

Table 1: Instruction encodings and control signals

3.1 Instruction Classification and Datapath

3.1.1 R-Type Instructions (Register)

Uses three registers: sources *rs*, *rt* and destination *rd*. The ALU performs the operation indicated by *funct*.

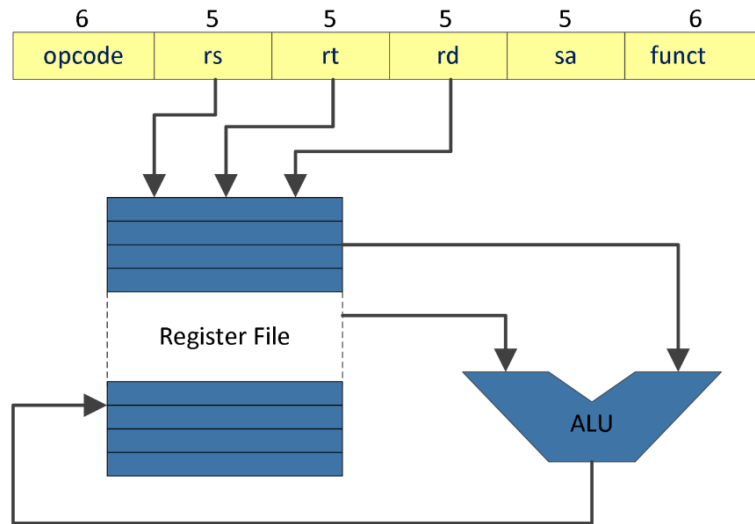


Figure 1: Datapath for R-Type instructions

3.1.2 I-Type Instructions (Immediate)

Uses a 16-bit immediate constant, extended to 32 bits (sign or zero) before entering the ALU.

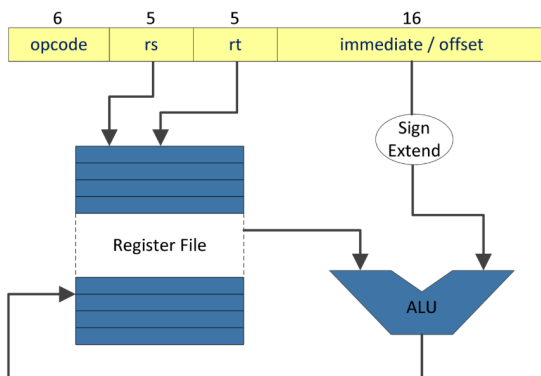


Figure 2: Sign Extend (addi)

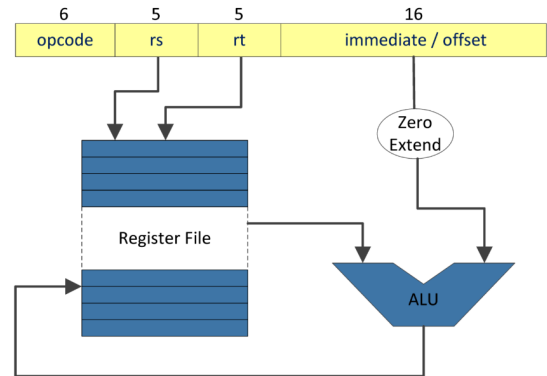


Figure 3: Zero Extend

Arithmetic and Logical with Immediate

Memory Access (lw and sw) The memory address is the sum of the base register and the sign-extended offset.

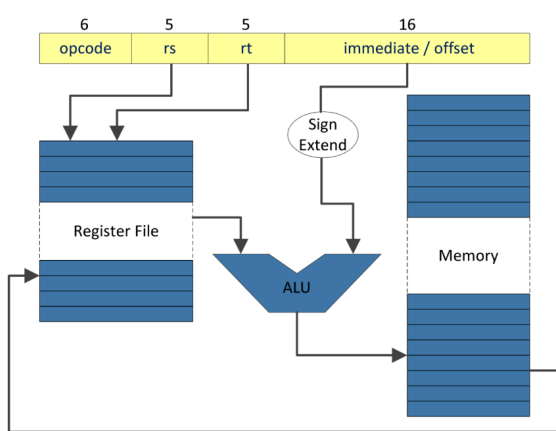


Figure 4: Load (lw)

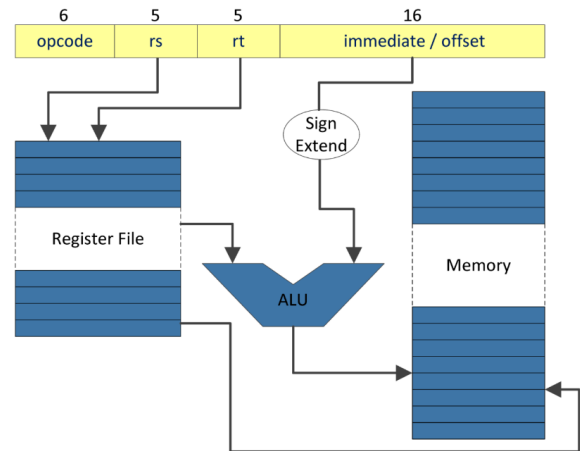


Figure 5: Store (sw)

Conditional Branch (beq, bne) The branch decision is based on checking operand equality in the ALU.

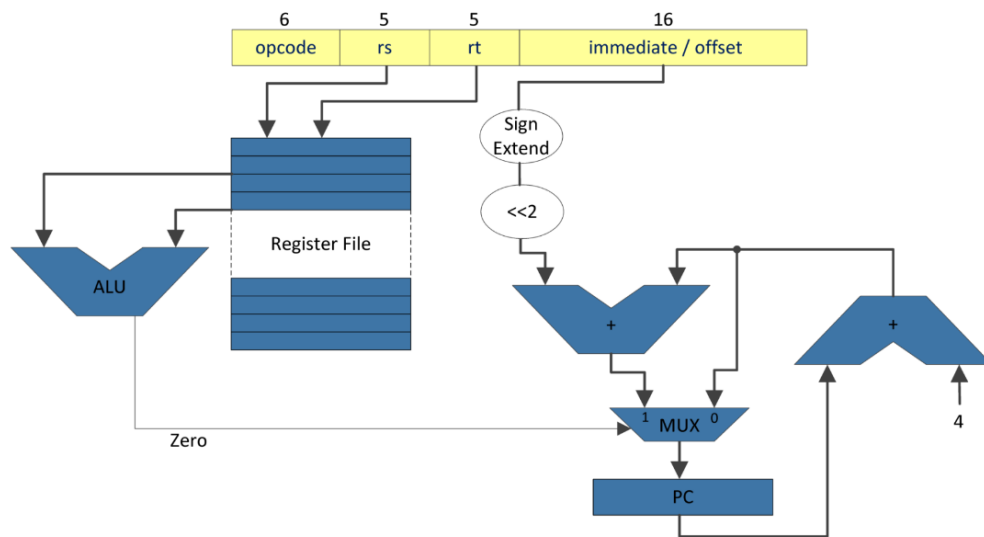


Figure 6: Branch instructions

3.1.3 J-Type Instructions (Jump)

Directly modifies the Program Counter with the provided address.

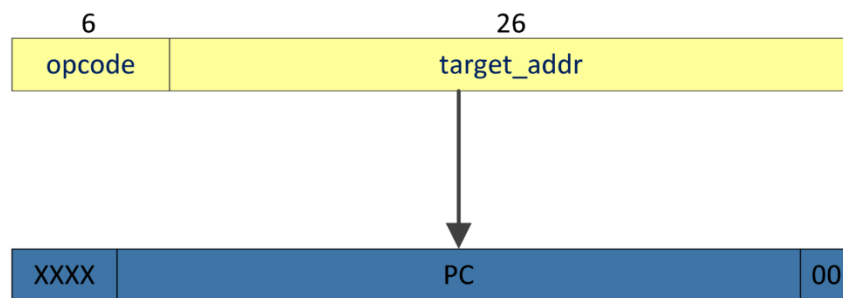


Figure 7: Jump instructions

4 Hardware Components and Architecture

4.1 Processor Block Diagram

The diagram illustrates the interconnection of the IFetch, IDecode, EX, and MEM modules. It was logically extended to support the `bne` instruction.

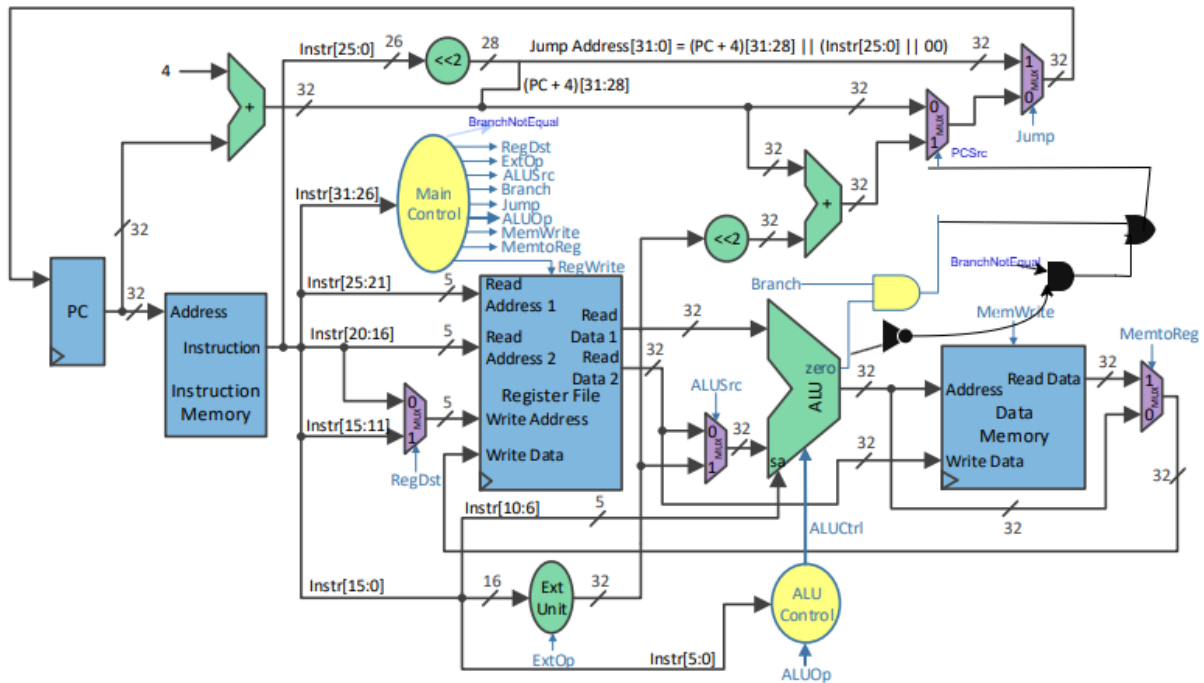


Figure 8: MIPS Single-Cycle Architecture

4.2 IFetch.vhd

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_UNSIGNED.ALL;
4
5  entity IFetch is
6      Port ( clk          : in  STD_LOGIC;
7            rst          : in  STD_LOGIC;
8            en           : in  STD_LOGIC;
9            BranchAddress : in  STD_LOGIC_VECTOR (31 downto 0);
10           JumpAddress  : in  STD_LOGIC_VECTOR (31 downto 0);
11           Jump         : in  STD_LOGIC;
12           PCSrc        : in  STD_LOGIC;
13           Instruction   : out STD_LOGIC_VECTOR (31 downto 0);
14           PC_plus_4     : out STD_LOGIC_VECTOR (31 downto 0));
15 end IFetch;
16
17 architecture Behavioral of IFetch is
18     signal PC      : STD_LOGIC_VECTOR(31 downto 0) := (others => '0');
19     signal Next_PC : STD_LOGIC_VECTOR(31 downto 0);
20     signal PC4      : STD_LOGIC_VECTOR(31 downto 0);
21     type rom_type is array (0 to 31) of std_logic_vector(31 downto 0);
22     signal ROM : rom_type := (
23         0 => B"000000_00000_00000_00100_00000_100000",
24         1 => B"001000_00000_01010_00000000000010000",
25         2 => B"100011_00100_00001_00000000000000000",
26         3 => B"100011_00100_00010_000000000000000100",
27         4 => B"100011_00100_00011_0000000000000001000",
28         5 => B"000000_00000_00000_00101_00000_100000",
29         6 => B"000000_00000_00000_00110_00000_100000",
30         7 => B"000000_00110_00011_00111_00000_101010",
31         8 => B"000100_00111_00000_00000000000001111",
32         9 => B"000000_00000_00110_01001_00010_000000",
33         10 => B"000000_01010_01001_01001_00000_100000",
34         11 => B"100011_01001_01000_00000000000000000",
35         12 => B"001000_00110_00110_00000000000000001",
36         13 => B"000000_01000_00001_00111_00000_101010",
37         14 => B"000101_00111_00000_1111111111111000",
38         15 => B"000000_00010_01000_00111_00000_101010",
39         16 => B"000101_00111_00000_11111111111110110",
40         17 => B"000000_00000_01000_00111_00000_101010",
41         18 => B"000100_00111_00000_11111111111110100",
42         19 => B"001000_01000_01001_1111111111111111",
43         20 => B"000000_01000_01001_01001_00000_100100",
44         21 => B"000101_01001_00000_11111111111110001",
45         22 => B"000000_00101_01000_00101_00000_100000",
46         23 => B"000010_0000000000000000000000000111",
47         24 => B"101011_00100_00101_00000000000001100",
48         25 => B"000010_0000000000000000000000011001",
49         others => B"000000_00000_00000_00000_00000_000000"
50     );
51 begin
52     process(clk, rst)
53     begin
54         if rst = '1' then
55             PC <= (others => '0');
56         elsif rising_edge(clk) then
57             if en = '1' then
58                 PC <= Next_PC;
```

```

59         end if;
60     end if;
61 end process;
62
63 PC4 <= PC + 4;
64 PC_plus_4 <= PC4;
65 Next_PC <= JumpAddress when Jump = '1' else
66         BranchAddress when PCSrc = '1' else
67         PC4;
68 Instruction <= ROM(conv_integer(PC(6 downto 2)));
69 end Behavioral;

```

4.3 IDecode.vhd

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_UNSIGNED.ALL;
4
5  entity IDecode is
6      Port ( clk          : in  STD_LOGIC;
7            en            : in  STD_LOGIC;
8            Instr         : in  STD_LOGIC_VECTOR (25 downto 0);
9            WD            : in  STD_LOGIC_VECTOR (31 downto 0);
10           RegWrite       : in  STD_LOGIC;
11           RegDst         : in  STD_LOGIC;
12           ExtOp          : in  STD_LOGIC;
13           RD1            : out STD_LOGIC_VECTOR (31 downto 0);
14           RD2            : out STD_LOGIC_VECTOR (31 downto 0);
15           Ext_Imm        : out STD_LOGIC_VECTOR (31 downto 0);
16           func           : out STD_LOGIC_VECTOR (5  downto 0);
17           sa             : out STD_LOGIC_VECTOR (4  downto 0));
18 end IDecode;
19
20 architecture Behavioral of IDecode is
21     type reg_array is array(0 to 31) of STD_LOGIC_VECTOR(31 downto 0);
22     signal reg_file : reg_array := (others => x"00000000");
23     signal WriteAddress : STD_LOGIC_VECTOR(4 downto 0);
24 begin
25     WriteAddress <= Instr(15 downto 11) when RegDst = '1' else Instr(20
26         downto 16);
27     process(clk)
28     begin
29         if rising_edge(clk) then
30             if en = '1' and RegWrite = '1' then
31                 reg_file(conv_integer(WriteAddress)) <= WD;
32             end if;
33         end if;
34     end process;
35
36     RD1 <= reg_file(conv_integer(Instr(25 downto 21)));
37     RD2 <= reg_file(conv_integer(Instr(20 downto 16)));
38     Ext_Imm(15 downto 0) <= Instr(15 downto 0);
39     Ext_Imm(31 downto 16) <= (others => Instr(15)) when ExtOp = '1' else (
40         others => '0');
41     func <= Instr(5 downto 0);
42     sa   <= Instr(10 downto 6);
43 end Behavioral;
```

4.4 EX.vhd

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_UNSIGNED.ALL;
4  use IEEE.NUMERIC_STD.ALL;
5
6  entity EX is
7      Port ( PC_plus_4      : in  STD_LOGIC_VECTOR (31 downto 0);
8            RD1             : in  STD_LOGIC_VECTOR (31 downto 0);
9            RD2             : in  STD_LOGIC_VECTOR (31 downto 0);
10           Ext_Imm          : in  STD_LOGIC_VECTOR (31 downto 0);
11           ALUSrc           : in  STD_LOGIC;
12           ALUOp            : in  STD_LOGIC_VECTOR (1 downto 0);
13           func             : in  STD_LOGIC_VECTOR (5 downto 0);
14           sa               : in  STD_LOGIC_VECTOR (4 downto 0);
15           BranchAddress    : out STD_LOGIC_VECTOR (31 downto 0);
16           ALURes           : out STD_LOGIC_VECTOR (31 downto 0);
17           Zero             : out STD_LOGIC);
18 end EX;
19
20 architecture Behavioral of EX is
21     signal ALUCtrl : STD_LOGIC_VECTOR(2 downto 0);
22     signal A, B     : STD_LOGIC_VECTOR(31 downto 0);
23     signal ALU_Out  : STD_LOGIC_VECTOR(31 downto 0);
24 begin
25     BranchAddress <= PC_plus_4 + (Ext_Imm(29 downto 0) & "00");
26     A <= RD1;
27     B <= Ext_Imm when ALUSrc = '1' else RD2;
28
29     process(ALUOp, func)
30     begin
31         case ALUOp is
32             when "00" => ALUCtrl <= "000";
33             when "01" => ALUCtrl <= "001";
34             when "10" =>
35                 case func is
36                     when "100000" => ALUCtrl <= "000";
37                     when "100100" => ALUCtrl <= "010";
38                     when "101010" => ALUCtrl <= "011";
39                     when "000000" => ALUCtrl <= "100";
40                     when others    => ALUCtrl <= "000";
41                 end case;
42             when others => ALUCtrl <= "000";
43         end case;
44     end process;
45
46     process(A, B, ALUCtrl, sa)
47     begin
48         case ALUCtrl is
49             when "000" => ALU_Out <= A + B;
50             when "001" => ALU_Out <= A - B;
51             when "010" => ALU_Out <= A and B;
52             when "011" =>
53                 if signed(A) < signed(B) then ALU_Out <= x"00000001";
54                 else ALU_Out <= x"00000000"; end if;
55             when "100" => ALU_Out <= to_stdlogicvector(to_bitvector(B) sll
56                 conv_integer(sa));
57             when others => ALU_Out <= (others => '0');
58         end case;
```

```
58     end process;  
59  
60     ALURes <= ALU_Out;  
61     Zero <= '1' when ALU_Out = x"00000000" else '0';  
62 end Behavioral;
```

4.5 MEM.vhd

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_UNSIGNED.ALL;
4
5  entity MEM is
6      Port ( clk          : in  STD_LOGIC;
7            en            : in  STD_LOGIC;
8            MemWrite      : in  STD_LOGIC;
9            ALUResin      : in  STD_LOGIC_VECTOR (31 downto 0);
10           RD2           : in  STD_LOGIC_VECTOR (31 downto 0);
11           MemData       : out STD_LOGIC_VECTOR (31 downto 0);
12           ALUResout     : out STD_LOGIC_VECTOR (31 downto 0));
13 end MEM;
14
15 architecture Behavioral of MEM is
16     type ram_type is array (0 to 63) of STD_LOGIC_VECTOR(31 downto 0);
17     signal RAM : ram_type := (
18         0 => x"00000002", -- RAM[0]: a = 2
19         1 => x"0000005A", -- RAM[4]: b = 90
20         2 => x"00000007", -- RAM[8]: n = 7
21         3 => x"00000000", -- RAM[12]: sum = 0
22         4 => x"00000000", -- RAM[16]: x[0] = 0
23         5 => x"00000001", -- RAM[20]: x[1] = 1
24         6 => x"00000002", -- RAM[24]: x[2] = 2
25         7 => x"00000003", -- RAM[28]: x[3] = 3
26         8 => x"00000100", -- RAM[32]: x[4] = 256
27         9 => x"0000000E", -- RAM[36]: x[5] = 14
28         10 => x"00000010", -- RAM[40]: x[6] = 16
29         others => x"00000000"
30     );
31
32 begin
33     process(clk)
34     begin
35         if rising_edge(clk) then
36             if en = '1' and MemWrite = '1' then
37                 RAM(conv_integer(ALUResin(7 downto 2))) <= RD2;
38             end if;
39         end if;
40     end process;
41     MemData <= RAM(conv_integer(ALUResin(7 downto 2)));
42     ALUResout <= ALUResin;
43 end Behavioral;
```

4.6 UC.vhd

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity UC is
5      Port ( Instr_opcode : in  STD_LOGIC_VECTOR (5 downto 0);
6            RegDst       : out STD_LOGIC;
7            ExtOp        : out STD_LOGIC;
8            ALUSrc       : out STD_LOGIC;
9            Branch       : out STD_LOGIC;
10           BranchNotEq  : out STD_LOGIC;
11           Jump         : out STD_LOGIC;
12           ALUOp        : out STD_LOGIC_VECTOR (1 downto 0);
13           MemWrite     : out STD_LOGIC;
14           MemtoReg     : out STD_LOGIC;
15           RegWrite     : out STD_LOGIC);
16 end UC;
17
18 architecture Behavioral of UC is
19 begin
20     process(Instr_opcode)
21     begin
22         RegDst <= '0'; ExtOp <= '0'; ALUSrc <= '0'; Branch <= '0';
23         BranchNotEq <= '0'; Jump <= '0'; MemWrite <= '0'; MemtoReg <= '0';
24         RegWrite <= '0'; ALUOp <= "00";
25
26         case Instr_opcode is
27             when "000000" => RegDst <= '1'; RegWrite <= '1'; ALUOp <= "10";
28             when "001000" => ExtOp <= '1'; ALUSrc <= '1'; RegWrite <= '1';
29                 ALUOp <= "00";
30             when "100011" => ExtOp <= '1'; ALUSrc <= '1'; MemtoReg <= '1';
31                 RegWrite <= '1'; ALUOp <= "00";
32             when "101011" => ExtOp <= '1'; ALUSrc <= '1'; MemWrite <= '1';
33                 ALUOp <= "00";
34             when "000100" => ExtOp <= '1'; Branch <= '1'; ALUOp <= "01";
35             when "000101" => ExtOp <= '1'; BranchNotEq <= '1'; ALUOp <= "01"
36                 ;
37             when "000010" => Jump <= '1';
38             when others => null;
39         end case;
40     end process;
41 end Behavioral;
```


4.7 test_env.vhd

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_UNSIGNED.ALL;
4
5  entity test_env is
6      Port ( clk : in  STD_LOGIC;
7            btn : in  STD_LOGIC_VECTOR (4 downto 0);
8            sw  : in  STD_LOGIC_VECTOR (15 downto 0);
9            led : out STD_LOGIC_VECTOR (15 downto 0);
10           an  : out STD_LOGIC_VECTOR (7  downto 0);
11           cat : out STD_LOGIC_VECTOR (6  downto 0));
12 end test_env;
13
14 architecture Behavioral of test_env is
15
16     component IFetch is
17         Port ( clk, rst, en : in STD_LOGIC;
18               BranchAddress, JumpAddress : in STD_LOGIC_VECTOR (31 downto
19                   0);
20               Jump, PCSrc : in STD_LOGIC;
21               Instruction, PC_plus_4 : out STD_LOGIC_VECTOR (31 downto 0));
22     end component;
23
24     component UC is
25         Port ( Instr_opcode : in STD_LOGIC_VECTOR (5 downto 0);
26               RegDst, ExtOp, ALUSrc, Branch, BranchNotEq, Jump : out
27                   STD_LOGIC;
28               ALUOp : out STD_LOGIC_VECTOR (1 downto 0);
29               MemWrite, MemtoReg, RegWrite : out STD_LOGIC);
30     end component;
31
32     component IDecode is
33         Port ( clk, en : in STD_LOGIC;
34               Instr : in STD_LOGIC_VECTOR (25 downto 0);
35               WD : in STD_LOGIC_VECTOR (31 downto 0);
36               RegWrite, RegDst, ExtOp : in STD_LOGIC;
37               RD1, RD2, Ext_Imm : out STD_LOGIC_VECTOR (31 downto 0);
38               func : out STD_LOGIC_VECTOR (5 downto 0);
39               sa : out STD_LOGIC_VECTOR (4 downto 0));
40     end component;
41
42     component EX is
43         Port ( PC_plus_4, RD1, RD2, Ext_Imm : in STD_LOGIC_VECTOR (31 downto
44             0);
45               ALUSrc : in STD_LOGIC;
46               ALUOp : in STD_LOGIC_VECTOR (1 downto 0);
47               func : in STD_LOGIC_VECTOR (5 downto 0);
48               sa : in STD_LOGIC_VECTOR (4 downto 0);
49               BranchAddress, ALURes : out STD_LOGIC_VECTOR (31 downto 0);
50               Zero : out STD_LOGIC);
51     end component;
52
53     component MEM is
54         Port ( clk, en, MemWrite : in STD_LOGIC;
55               ALUResin, RD2 : in STD_LOGIC_VECTOR (31 downto 0);
56               MemData, ALUResout : out STD_LOGIC_VECTOR (31 downto 0));
57     end component;
```

```

56     component MPG is
57         Port ( enable : out STD_LOGIC;
58               btn, clk : in STD_LOGIC);
59     end component;
60
61     component SSD is
62         Port ( clk : in STD_LOGIC;
63               digits : in STD_LOGIC_VECTOR(31 downto 0);
64               an : out STD_LOGIC_VECTOR(7 downto 0);
65               cat : out STD_LOGIC_VECTOR(6 downto 0));
66     end component;
67
68     signal en, rst : STD_LOGIC;
69     signal Instruction, PC_plus_4 : STD_LOGIC_VECTOR(31 downto 0);
70     signal RegDst, ExtOp, ALUSrc, Branch, BranchNotEq, Jump, MemWrite,
71         MemtoReg, RegWrite : STD_LOGIC;
72     signal ALUOp : STD_LOGIC_VECTOR(1 downto 0);
73     signal RD1, RD2, Ext_Imm : STD_LOGIC_VECTOR(31 downto 0);
74     signal func : STD_LOGIC_VECTOR(5 downto 0);
75     signal sa : STD_LOGIC_VECTOR(4 downto 0);
76     signal BranchAddress, ALURes, MemData, ALUResout, WD, JumpAddress :
77         STD_LOGIC_VECTOR(31 downto 0);
78     signal Zero, PCSrc : STD_LOGIC;
79     signal digits_out : STD_LOGIC_VECTOR(31 downto 0);
80
81 begin
82
83     empege: MPG port map (enable => en, btn => btn(0), clk => clk);
84     rst <= btn(1);
85     display: SSD port map (clk => clk, digits => digits_out, an => an, cat
86         => cat);
87
88     inst_IFetch: IFetch port map (clk, rst, en, BranchAddress, JumpAddress,
89         Jump, PCSrc, Instruction, PC_plus_4);
90     inst_UC: UC port map (Instruction(31 downto 26), RegDst, ExtOp, ALUSrc,
91         Branch, BranchNotEq, Jump, ALUOp, MemWrite, MemtoReg, RegWrite);
92     inst_IDecode: IDecode port map (clk, en, Instruction(25 downto 0), WD,
93         RegWrite, RegDst, ExtOp, RD1, RD2, Ext_Imm, func, sa);
94     inst_EX: EX port map (PC_plus_4, RD1, RD2, Ext_Imm, ALUSrc, ALUOp, func,
95         sa, BranchAddress, ALURes, Zero);
96     inst_MEM: MEM port map (clk, en, MemWrite, ALURes, RD2, MemData,
97         ALUResout);
98
99     WD <= MemData when MemtoReg = '1' else ALUResout;
100    PCSrc <= (Branch and Zero) or (BranchNotEq and not Zero);
101    JumpAddress <= PC_plus_4(31 downto 28) & Instruction(25 downto 0) & "00"
102        ;
103
104    led(0) <= RegDst; led(1) <= ExtOp; led(2) <= ALUSrc; led(3) <= Branch;
105    led(4) <= BranchNotEq; led(5) <= Jump; led(6) <= MemWrite; led(7) <=
106        MemtoReg;
107    led(8) <= RegWrite; led(10 downto 9) <= ALUOp; led(15 downto 11) <= (
108        others => '0');
109
110    process(sw(7 downto 5), Instruction, PC_plus_4, RD1, RD2, Ext_Imm,
111        ALURes, MemData, WD)
112    begin
113        case sw(7 downto 5) is
114            when "000" => digits_out <= Instruction;
115            when "001" => digits_out <= PC_plus_4;

```

```
104         when "010" => digits_out <= RD1;
105         when "011" => digits_out <= RD2;
106         when "100" => digits_out <= Ext_Imm;
107         when "101" => digits_out <= ALURes;
108         when "110" => digits_out <= MemData;
109         when "111" => digits_out <= WD;
110         when others => digits_out <= (others => '0');
111     end case;
112 end process;
113
114 end Behavioral;
```

5 Machine Code and Execution

PC	Machine Code (Binary)	Hex	MIPS ASM
0	001000_00000_00001_00000000000000010	20010002	addi \$1, \$0, 2
1	001000_00000_00010_00000000001011010	2002005A	addi \$2, \$0, 90
2	001000_00000_00011_00000000000000111	20030007	addi \$3, \$0, 7
3	000000_00000_00000_00100_00000_100000	00002020	add \$4, \$0, \$0 (<i>la \$4, x</i>)
4	000000_00000_00000_00101_00000_100000	00002820	add \$5, \$0, \$0
5	000000_00000_00000_00110_00000_100000	00003020	add \$6, \$0, \$0
— <i>loop_start</i> : ($PC = 6$) —			
6	000000_00110_00011_00111_00000_101010	00C3382A	slt \$7, \$6, \$3
7	000100_00111_00000_00000000000001111	10E0000F	beq \$7, \$0, end (<i>+15</i>)
8	000000_00000_00110_01001_00010_000000	00064880	sll \$9, \$6, 2
9	000000_00100_01001_01001_00000_100000	00894820	add \$9, \$4, \$9
10	100011_01001_01000_00000000000000000	8D280000	lw \$8, 0(\$9)
11	001000_00110_00110_00000000000000001	20C60001	addi \$6, \$6, 1
12	000000_01000_00001_00111_00000_101010	0101382A	slt \$7, \$8, \$1
13	000101_00111_00000_11111111111111000	14E0FFF8	bne \$7, \$0, loop_start (<i>-8</i>)
14	000000_00010_01000_00111_00000_101010	0048382A	slt \$7, \$2, \$8
15	000101_00111_00000_11111111111110110	14E0FFF6	bne \$7, \$0, loop_start (<i>-10</i>)
16	000000_00000_01000_00111_00000_101010	0008382A	slt \$7, \$0, \$8
17	000100_00111_00000_11111111111110100	10E0FFF4	beq \$7, \$0, loop_start (<i>-12</i>)
18	001000_01000_01001_11111111111111111	2109FFFF	addi \$9, \$8, -1
19	000000_01000_01001_01001_00000_100100	01094824	and \$9, \$8, \$9
20	000101_01001_00000_11111111111110001	1520FFF1	bne \$9, \$0, loop_start (<i>-15</i>)
21	000000_00101_01000_00101_00000_100000	00A82820	add \$5, \$5, \$8
22	000010_000000000000000000000000110	08000006	j loop_start (<i>Jump target 6</i>)
— <i>end</i> : ($PC = 23$) —			
23	001000_00000_00010_00000000000001010	2002000A	addi \$v0, \$0, 10 (<i>\$v0 is \$2</i>)
24	000000_00000_00000_00000_00000_001100	0000000C	syscall (<i>generic syscall code</i>)

Table 2: Detailed mapping between source code and ROM memory

6 Execution Trace

In the first iteration, index $i = 0$, and the value read from memory is $x[0] = 0$.

Step	SW(7:5)	"000"	"001"	"010"	"011"	"100"	"101"	"110"	"111"	Only for branch	
	Instr (assembly)	Instr (hex)	PC+4	RD1	RD2	Ext_Imm	ALURes	MemData	WD	BranchAddr	JumpAddr
0	addi \$1, \$0, 2	20010002	00000004	00000000	00000000	00000002	00000002	XXXXXXXX	00000002	XXXXXXXX	XXXXXXXX
1	addi \$2, \$0, 90	2002005A	00000008	00000000	00000000	0000005A	0000005A	XXXXXXXX	0000005A	XXXXXXXX	XXXXXXXX
2	addi \$3, \$0, 7	20030007	0000000C	00000000	00000000	00000007	00000007	XXXXXXXX	00000007	XXXXXXXX	XXXXXXXX
3	add \$4, \$0, \$0	00002020	00000010	00000000	00000000	XXXXXXXX	00000000	XXXXXXXX	00000000	XXXXXXXX	XXXXXXXX
4	add \$5, \$0, \$0	00002820	00000014	00000000	00000000	XXXXXXXX	00000000	XXXXXXXX	00000000	XXXXXXXX	XXXXXXXX
5	add \$6, \$0, \$0	00003020	00000018	00000000	00000000	XXXXXXXX	00000000	XXXXXXXX	00000000	XXXXXXXX	XXXXXXXX
6	slt \$7, \$6, \$3	00C3382A	0000001C	00000000	00000007	XXXXXXXX	00000001	XXXXXXXX	00000001	XXXXXXXX	XXXXXXXX
7	beq \$7, \$0, end	10E0000F	00000020	00000001	00000000	0000000F	00000001	XXXXXXXX	XXXXXXXX	0000005C	XXXXXXXX
8	sll \$9, \$6, 2	00064880	00000024	XXXXXXXX	00000000	XXXXXXXX	00000000	XXXXXXXX	00000000	XXXXXXXX	XXXXXXXX
9	add \$9, \$4, \$9	00894820	00000028	00000000	00000000	XXXXXXXX	00000000	XXXXXXXX	00000000	XXXXXXXX	XXXXXXXX
10	lw \$8, 0(\$9)	8D280000	0000002C	00000000	00000000	00000000	00000000	00000000	00000000	XXXXXXXX	XXXXXXXX
11	addi \$6, \$6, 1	20C60001	00000030	00000000	XXXXXXXX	00000001	00000001	XXXXXXXX	00000001	XXXXXXXX	XXXXXXXX
12	slt \$7, \$8, \$1	0101382A	00000034	00000000	00000002	XXXXXXXX	00000001	XXXXXXXX	00000001	XXXXXXXX	XXXXXXXX
13	bne \$7, \$0, loop	14E0FFF8	00000038	00000001	00000000	FFFFFFF8	00000001	XXXXXXXX	XXXXXXXX	00000018	XXXXXXXX
— Second iteration (resume loop_start) —											
14	slt \$7, \$6, \$3	00C3382A	0000003C	00000001	00000007	XXXXXXXX	00000001	XXXXXXXX	00000001	XXXXXXXX	XXXXXXXX

Table 3: Execution trace (First full iteration)

Execution trace note: The table reflects the datapath state at the execution of each instruction. The "XXXXXXXX" values represent "Don't Care" signals for that respective instruction.

7 FPGA Synthesis

The project synthesis in the Vivado environment completed successfully, generating the `test_env.bit` file. The configuration was programmed onto the FPGA.